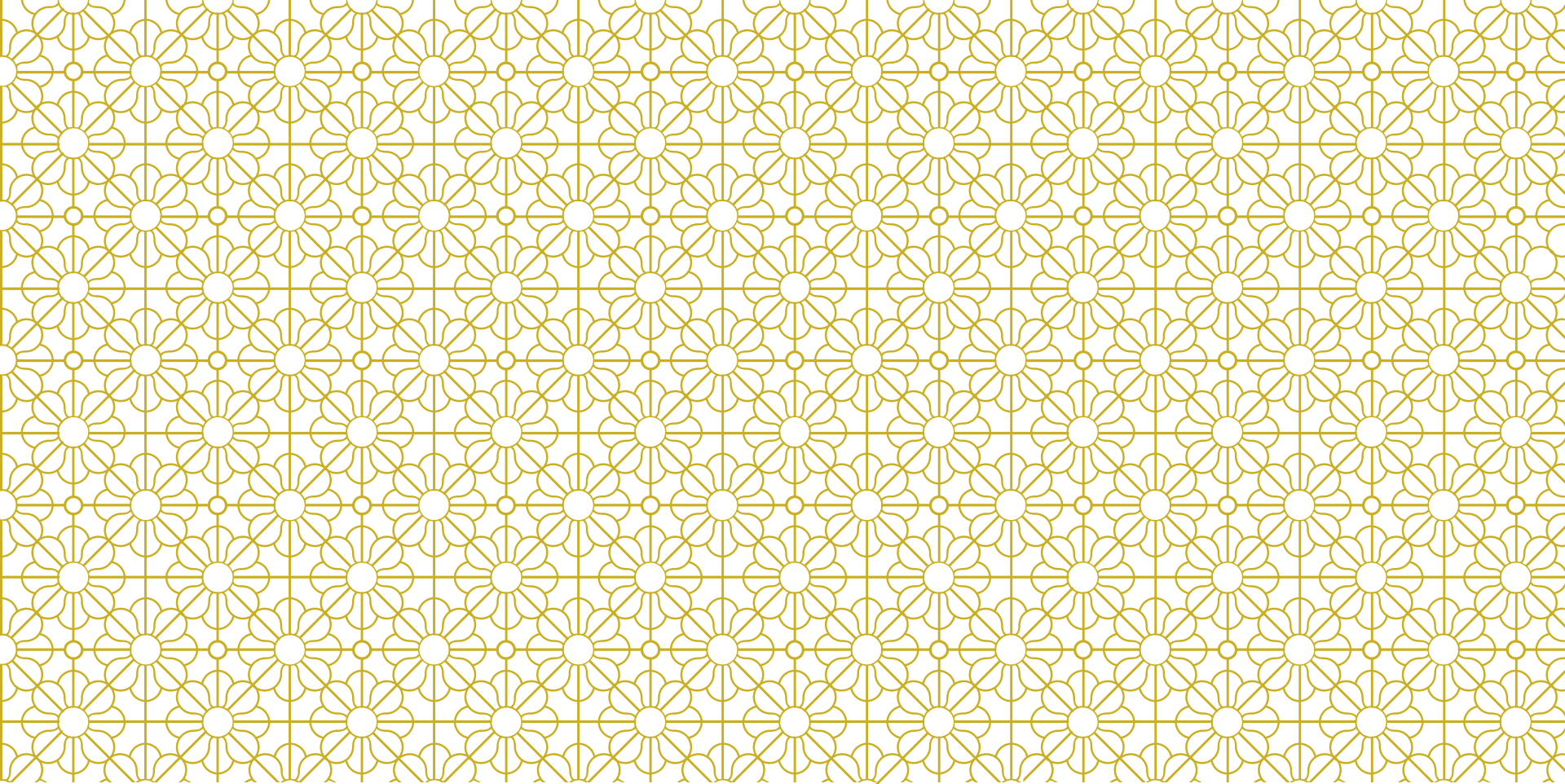
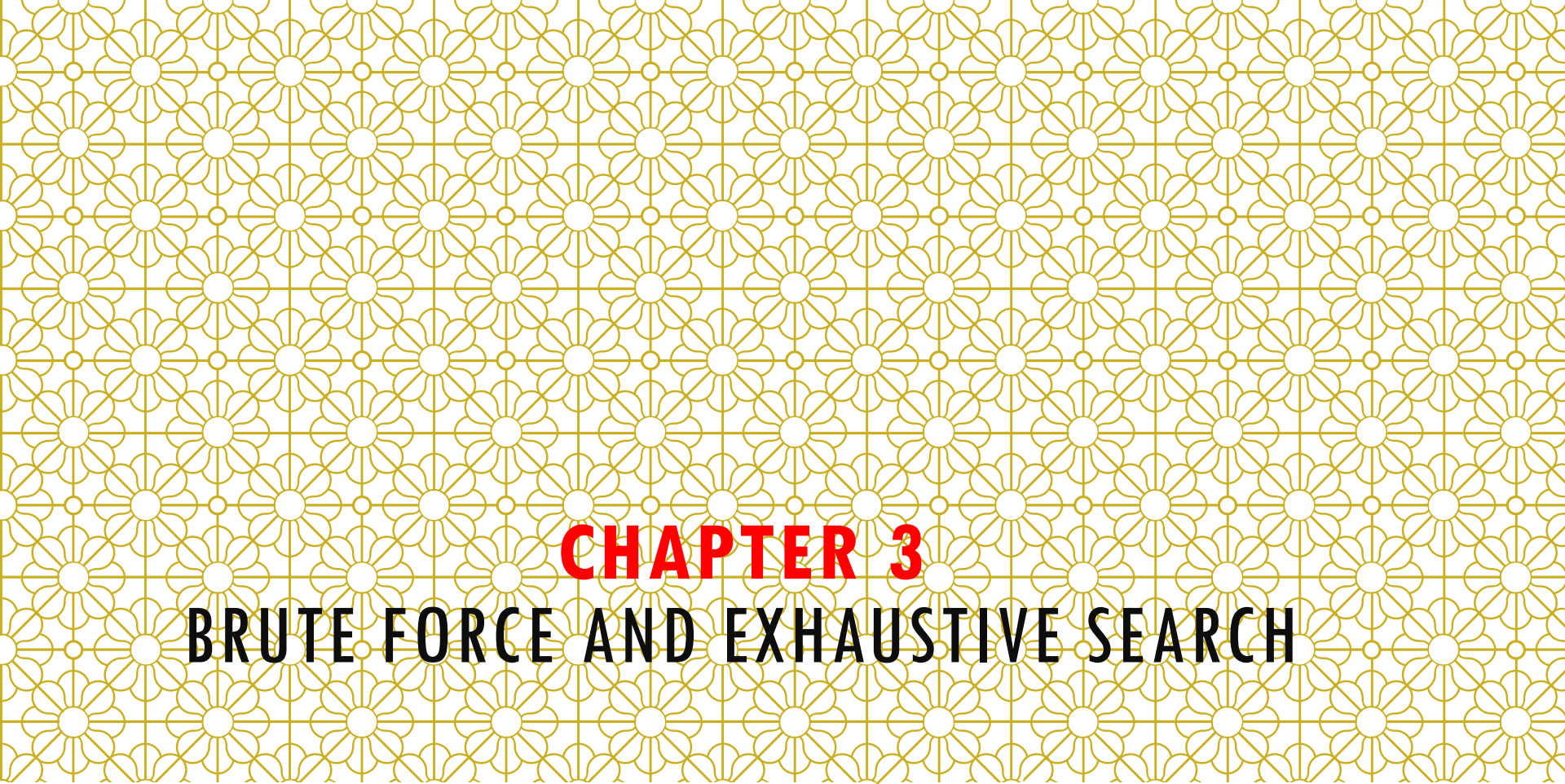




ALGORITHMS ANALYSIS AND DESIGN

By : Dr. Basem
Alijla



CHAPTER 3

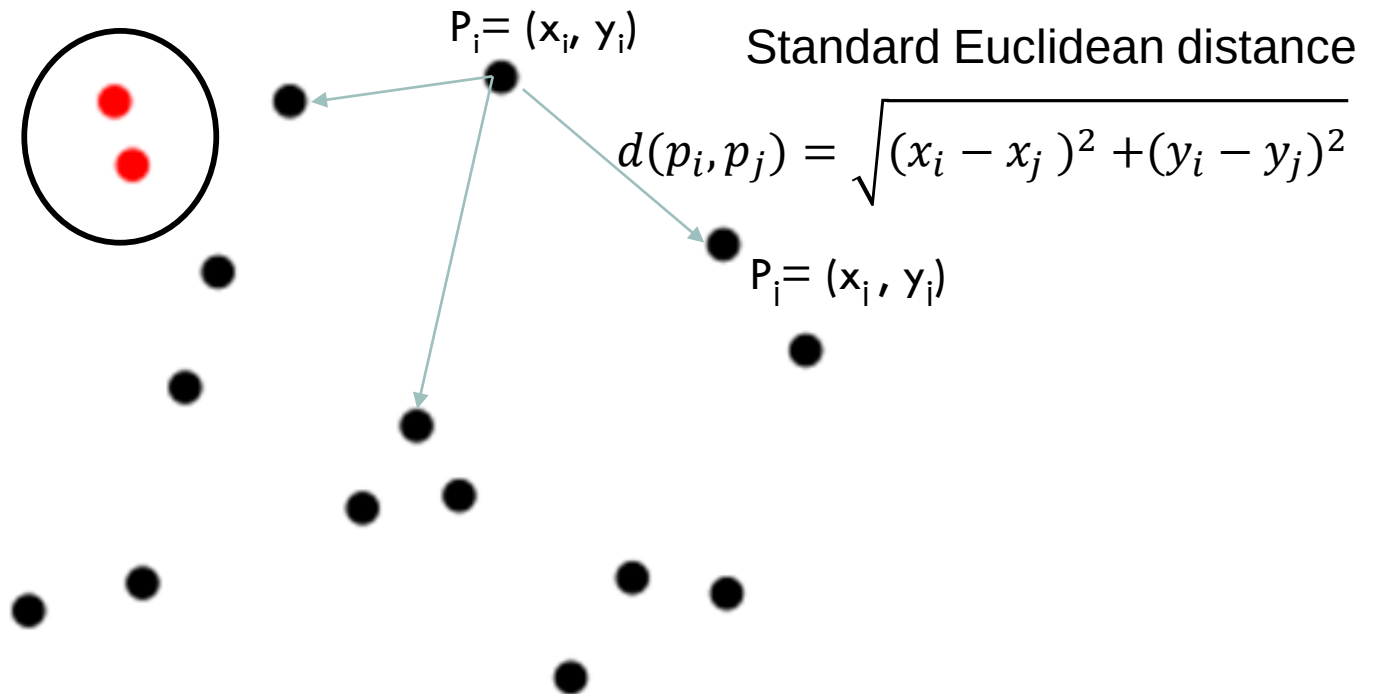
BRUTE FORCE AND EXHAUSTIVE SEARCH

OUTLINE

- **Introduction to Brute-Force approach**
- **Selection Sort and Bubble Sort**
- **Sequential Search and Brute-Force String Matching**
- **Closest-Pair Problem by Brute Force**
- **Exhaustive Search**
- **Depth-First Search and Breadth-First Search**

CLOSEST PAIR

Problem statement: To find out the two closest points in a set of n points in the two-dimensional cartesian plane.



CLOSEST PAIR

Closest Point pair in the Real world.

- ❖ Air traffic control system (closest airplanes than may collide)
- ❖ Closest post office (to close one of them)
- ❖ Database records (similar records)
- ❖ Clustering analysis based on similarity measure

CLOSEST PAIR ALGORITHM

Algorithm BruteForceClosestPoints(P)

// P is list of points

$dmin \leftarrow \infty$

for $i \leftarrow 1$ **to** $n-1$ **do**

for $j \leftarrow i+1$ **to** n **do**

$d \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$

if $d < dmin$ **then**

$dmin \leftarrow d; index1 \leftarrow i; index2 \leftarrow j$

return $index1, index2$

Efficiency : (n^2)

OUTLINE

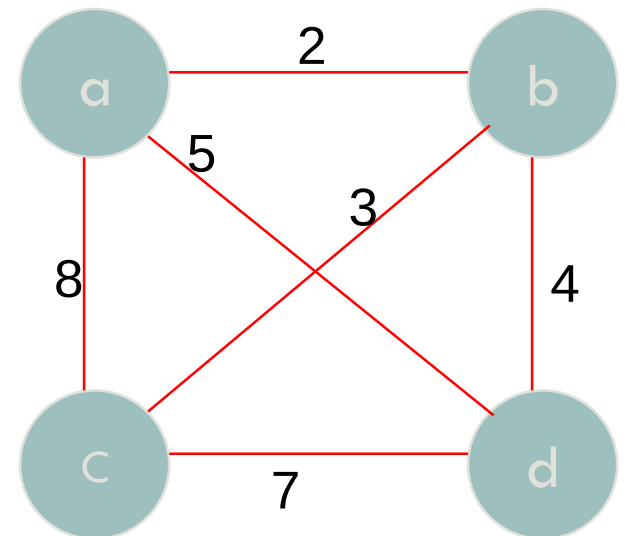
- **Introduction to Brute-Force approach**
- **Selection Sort and Bubble Sort**
- **Sequential Search and Brute-Force String Matching**
- **Closest-Pair Problem by Brute Force**
- **Exhaustive Search**
- **Depth-First Search and Breadth-First Search**

EXHAUSTIVE SEARCH

- ❖ A brute force approach to combinatorial problems (which require generation of **permutations**, **Combinations**, or **subsets**) (e.g., TSP)

A simple instance of

Traveling salesman problem (TSP)



EXHAUSTIVE SEARCH

- ❖ generate a list of all potential solutions to the problem in a systematic manner. (Grow exponentially with the instance size).
- ❖ evaluate potential solutions one by one, disqualifying infeasible ones.
- ❖ For an optimization problem, keeping track of the best one found so far when search ends, announce the solution(s) found

PROBLEMS SOLVED BY EXHAUSTIVE SEARCH

- 1) Traveling salesman problem (TSP)**
- 2) knapsack problem (KSP).**

TRAVELING SALESMAN PROBLEM (TSP)

Traveling salesman problem (TSP)

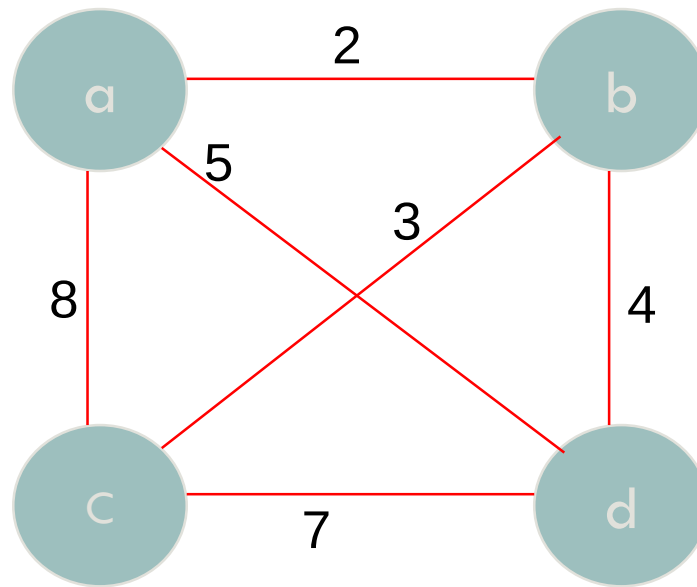
Given n cities with known distances between each pair; find the shortest tour that passes through all the cities exactly once before returning to the starting city.

Find shortest Hamiltonian circuit in a weighted connected graph.

Hamiltonian circuit : is a cycle that passes through all the vertices of the graph exactly once. (William Rowan Hamilton (1805–1865))

TRAVELING SALESMAN PROBLEM (TSP)

A simple instance of Traveling salesman problem (TSP)



TSP BY EXHAUSTIVE SEARCH

How to Solve TSP by exhaustive search

- get all the tours by generating **all the permutations** of $(n - 1)$ intermediate cities. $(n-1)!$
- Compute the length of visible tours
- Find the shortest among them.

TSP BY EXHAUSTIVE SEARCH

Tour	Cost
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2+3+7+5 = 17$ ← optimal
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2+4+7+8 = 21$
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$8+3+4+5 = 20$
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$8+7+4+2 = 21$
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$5+4+3+8 = 20$
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$5+7+3+2 = 17$ ← optimal

Efficiency : $\frac{1}{2} (n-1)!$

KNAPSACK PROBLEM

Given n items of weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack of capacity W .

Find the most valuable subset of the items that fit into the knapsack.

A transport plane has to deliver the most valuable set of items to a remote location without exceeding its capacity

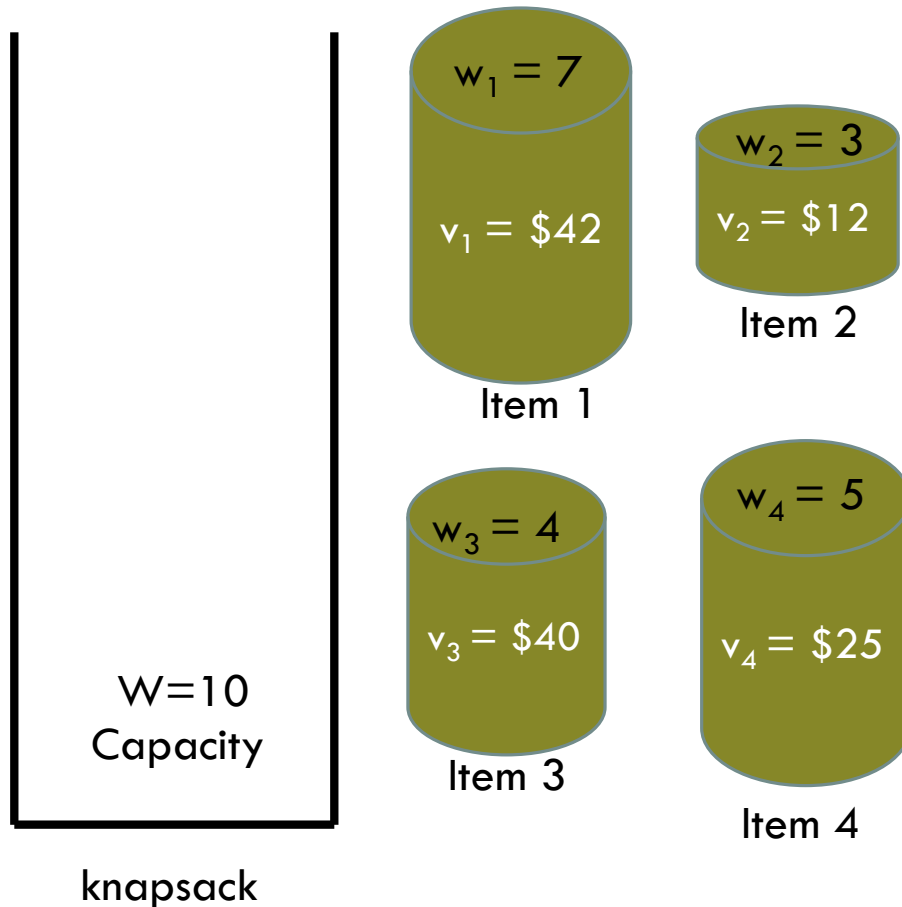
BRUTE FORCE FOR KNAPSACK PROBLEM

How can we solve it ?

- Generate all possible subsets of the n items.
- Compute total weight of each subset to identify feasible subsets.
- Find the subset of the largest value

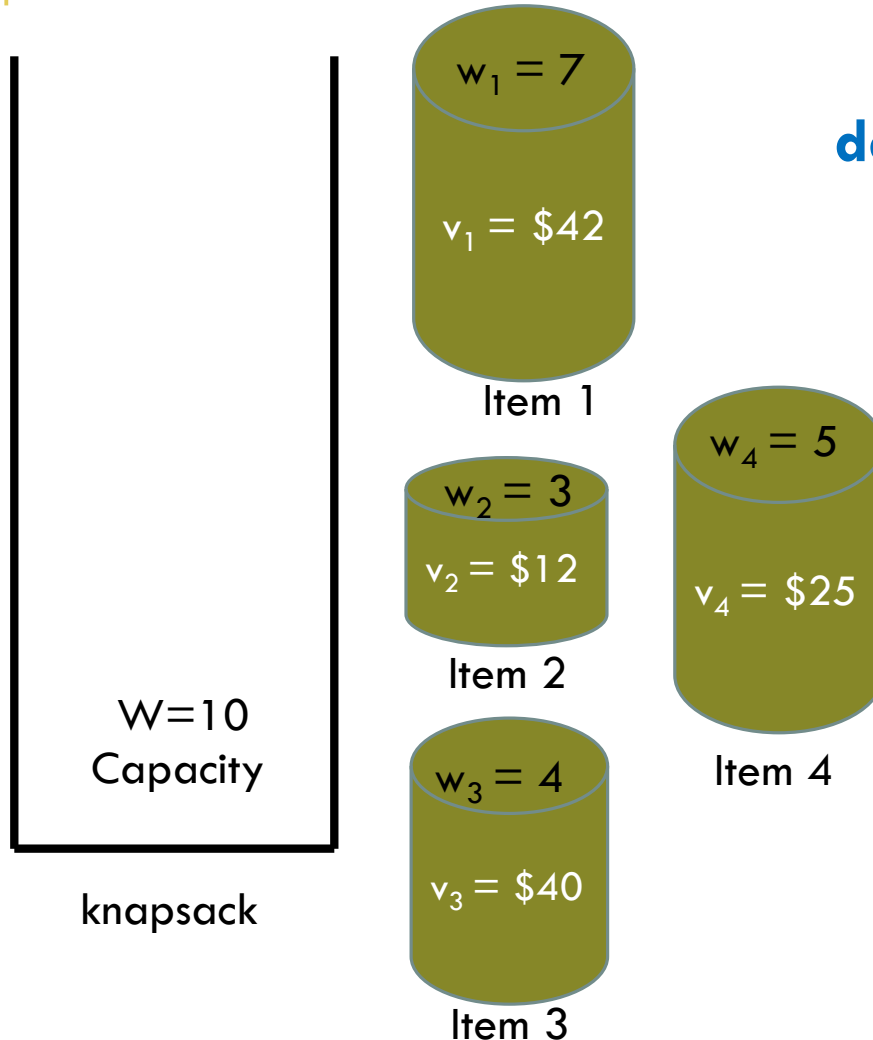
What is the time efficiency ? $O(2^n)$

BRUTE FORCE FOR KNAPSACK PROBLEM



subset	weight	value
$\emptyset$	0	\$0
{1}	7	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1,2}	10	\$54
{1,3}	11	!feasible
{1,4}	12	!feasible
{2,3}	7	\$52
{2,4}	8	\$37
{3,4}	9	\$65
{1,2,3}	14	!feasible
{1,2,4}	15	!feasible
{1,3,4}	16	!feasible
{2,3,4}	12	!feasible
{1,2,3,4}	19	!feasible

BRUTE FORCE FOR KNAPSACK PROBLEM



How about taking items in decreasing order of value/weight?

Item3: \$10/unit

Item1: \$6/unit

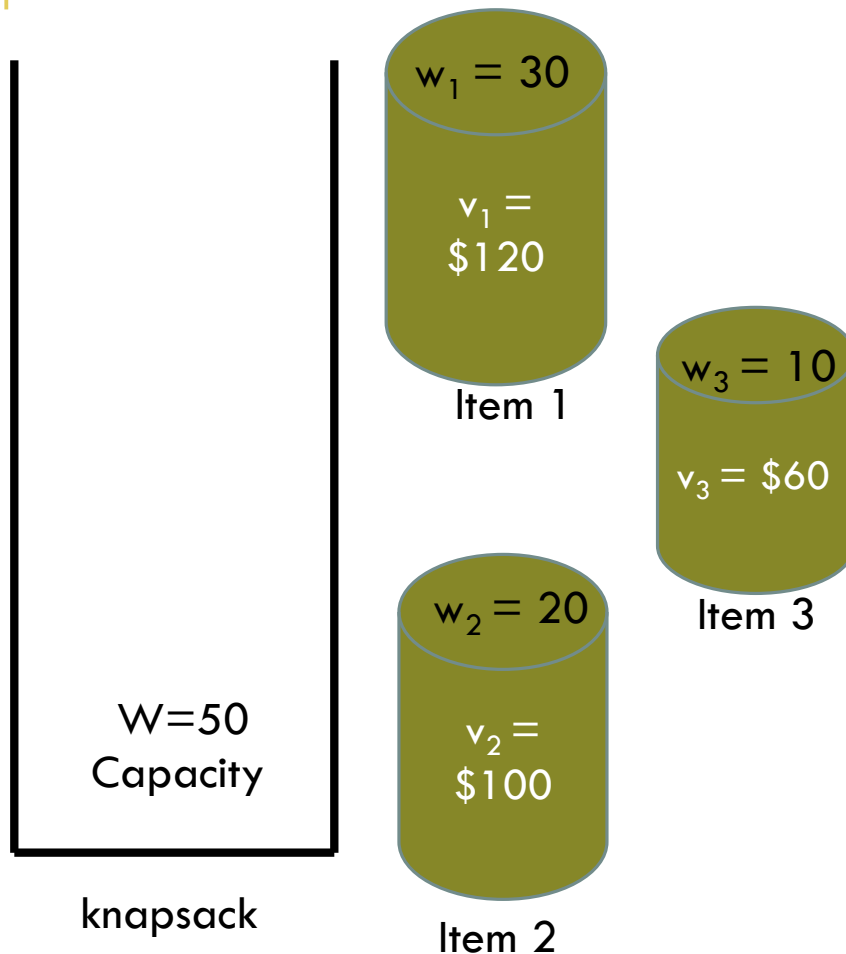
Item4: \$5/unit

Item2: \$4/unit

{Item3, Item4} = \$40+\$25 = \$65

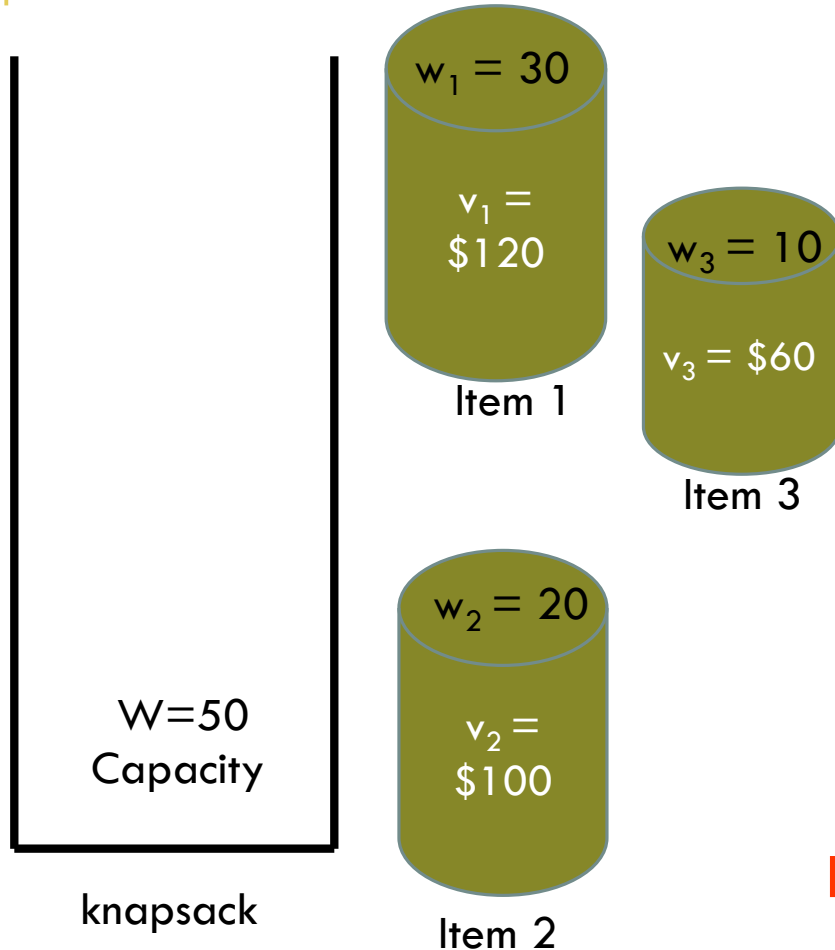
Works for this example! 😊

BRUTE FORCE FOR KNAPSACK PROBLEM



subset	weight	value
$\emptyset$	0	\$0
{1}	30	\$120
{2}	20	\$100
{3}	10	\$60
{1,2}	50	\$220
{1,3}	40	\$180
{2,3}	30	\$160
{1,2,3}	60	!feasible

BRUTE FORCE FOR KNAPSACK PROBLEM



How about taking items in decreasing order of value/weight?

Item3: \$6/unit

Item 2: \$5/unit

Item1: \$4/unit

{Item3, Item2} = \$60+\$100 = \$160

{Item3, Item1} = \$60+\$120 = \$180

{Item2, Item1} = \$100+\$120 = \$220

Doesn't work for this example ☹️

TSP BY EXHAUSTIVE SEARCH

For both Traveling Salesman and Knapsack problems, exhaustive search gives exponential time complexity.

TSP and Knapsack problems are best-known examples of NP-hard problems (i.e. no serial algorithm exists that runs in time polynomial in n , only in time exponential in n , and it is widely believed that no polynomial time algorithm exists.